

**LAPORAN PRAKTIKUM STRUKTUR
DATA**

**MODUL
GRAPH**



Disusun Oleh :

NAMA : Ayu Setyaning Tyas

NIM : 103112430119

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Dasar Teori

Dalam matematika dan ilmu komputer, teori graf adalah studi tentang graf, yaitu struktur matematika yang digunakan untuk memodelkan hubungan berpasangan antar objek. Graf dalam konteks ini terdiri dari simpul (juga disebut node atau titik) yang dihubungkan oleh sisi (juga disebut busur, tautan, atau garis). Perbedaan dibuat antara graf tak berarah, di mana sisi menghubungkan dua simpul secara simetris, dan graf berarah, di mana sisi menghubungkan dua simpul secara asimetris.

Fungsi grafik digunakan untuk mencari jalur terpendek, memodelkan jaringan kompleks seperti jaringan sosial, serta membantu menyelesaikan masalah optimasi, misalnya menentukan rute paling optimal pada algoritma Traveling Salesman.

Graph banyak digunakan dalam kehidupan sehari-hari, seperti pada media sosial untuk menghubungkan pengguna dan memfasilitasi interaksi, navigasi dan peta untuk mencari rute terpendek, serta sistem rekomendasi produk yang menyarankan barang berdasarkan kesamaan preferensi. Selain itu, graph juga dimanfaatkan dalam perencanaan transportasi dan rute pengiriman agar lebih efisien, perancangan sirkuit elektronik, serta pencarian informasi dengan menghubungkan halaman web dalam mesin pencari.

B. Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

Code *grafguided.c*

```
#ifndef GRAFGUIDED_H_INCLUDED
#define GRAFGUIDED_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infoGraph;

struct ElmNode;
struct ElmEdge;

typedef ElmNode *adrNode;
typedef ElmEdge *adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct ElmEdge {
    adrNode node;
    adrEdge next;
};
```

```

struct Graph {
    adrNode first;
};

void createGraph(Graph &G);
adrNode AllocatedNode(infoGraph X);
adrEdge AllocatedEdge(adrNode N);

void insertNode(Graph &G, infoGraph X);
void FindNode(Graph G, infoGraph X);

void ConnectNode(Graph &G, infoGraph A, infoGraph B);

void printInfoGraph(Graph G);

void ResetVisited(Graph &G);
void printDFS(Graph &G, adrNode N);
void printBFS(Graph &G, adrNode N);

#endif

```

Code *grafguided.cpp*

```

#include "grafguided.h"
#include <queue>
#include <stack>

void createGraph(Graph &G)
{
    G.first = NULL;
}

adrNode AllocatedNode(infoGraph X)
{
    adrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
    P->firstEdge = NULL;
    P->next = NULL;
    return P;
}

adrEdge AllocatedEdge(adrNode N)
{
    adrEdge P = new ElmEdge;
    P->node = N;
    P->next = NULL;
}

```

```

    return P;
}

void InsertNode(Graph &G, infoGraph X)
{
    adrNode P = AllocatedNode(X);
    P->next = G.first;
    G.first = P;
}

adrNode findNode(Graph G, infoGraph X)
{
    adrNode P = G.first;
    while (P != NULL) {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(Graph &G, infoGraph A, infoGraph B)
{
    adrNode N1 = findNode(G, A);
    adrNode N2 = findNode(G, B);

    if (N1 == NULL || N2 == NULL) {
        cout << "Node tidak ditemukan!\n";
        return;
    }

    // buat edge dari N1 ke N2
    adrEdge E1 = AllocatedEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;

    // karena undirected -> buat edge balik
    adrEdge E2 = AllocatedEdge(N1);
    E2->next = N2->firstEdge;
    N2->firstEdge = E2;
}

void printInfoGraph(Graph G)
{
    adrNode P = G.first;
    while (P != NULL)
    {
        cout << P->info << "->";
    }
}

```

```

        adrEdge E = P->firstEdge;
        while (E != NULL)
        {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

void ResetVisited(Graph &G)
{
    adrNode P = G.first;
    while (P != NULL)
    {
        P->visited = 0;
        P = P->next;
    }
}

void printDFS(Graph &G, adrNode N)
{
    if (N == NULL)
        return;

    N->visited = 1;
    cout << N->info << " ";

    adrEdge E = N->firstEdge;
    while (E != NULL)
    {
        if (E->node->visited == 0)
        {
            printDFS(G, E->node);
        }
        E = E->next;
    }
}

void printBFS(Graph &G, adrNode N)
{
    if (N == NULL)
        return;

    queue<adrNode> Q;
    Q.push(N);

```

```

while (!Q.empty())
{
    adrNode curr = Q.front();
    Q.pop();

    if (curr->visited == 0)
    {
        curr->visited = 1;
        cout << curr->info << " ";

        adrEdge E = curr->firstEdge;
        while (E != NULL)
        {
            if (E->node->visited == 0)
            {
                Q.push(E->node);
            }
            E = E->next;
        }
    }
}
}

```

Code *maingraf.c*

```

#include "grafguided.h"
#include "grafguided.cpp"
#include <iostream>
using namespace std;

int main() {
    Graph G;
    createGraph(G);

    //Tambah node
    InsertNode(G, 'A');
    InsertNode(G, 'B');
    InsertNode(G, 'C');
    InsertNode(G, 'D');
    InsertNode(G, 'E');

    // hubungkan node (graf tidak berarah)
    ConnectNode(G, 'A', 'B');
    ConnectNode(G, 'A', 'C');
    ConnectNode(G, 'B', 'D');
    ConnectNode(G, 'C', 'E');

    cout << "=== Struktur Graph ===" << endl;
}

```

```

    printInfoGraph(G);

    cout << "\n=== Traversal DFS dari node A ===" << endl;
    ResetVisited(G);
    printDFS(G, findNode(G, 'A'));

    cout << "\n\n=== Traversal BFS dari node A ===" << endl;
    ResetVisited(G);
    printBFS(G, findNode(G, 'A'));

    cout << endl;
    return 0;
}

```

Screenshoot Output

```

PS D:\Cpp\laprak 12> cd "d:\Cpp\laprak 12\" ; if ($?) { g++ mainGraph.cpp -o mainGraph } ; if ($?) { .\mainGraph }
=== Struktur Graph ===
E->C
D->B
C->E A
B->D A
A->C B

=== Traversal DFS dari node A ===
A C E B D

=== Traversal BFS dari node A ===
A C B E D
PS D:\Cpp\laprak 12> cd "d:\Cpp\laprak 12\" ; if ($?) { g++ mainGraph.cpp -o mainGraph } ; if ($?) { .\mainGraph }

```

C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

Unguided 1

Code *Graph.h*

```

#ifndef GRAPH_H_INCLUDED
#define GRAPH_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infoGraph;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* adrNode;
typedef ElmEdge* adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
}

```

```

};

struct ElmEdge {
    adrNode node;
    adrEdge next;
};

struct Graph {
    adrNode first;
};

void createGraph(Graph &G);
adrNode AllocatedNode(infoGraph X);
adrEdge AllocatedEdge(adNode N);

void InsertNode(Graph &G, infoGraph X);
adrNode findNode(Graph G, infoGraph X);

void ConnectNode(Graph &G, infoGraph A, infoGraph B);

void PrintInfoGraph(Graph G);
void ResetVisited(Graph &G); // dipake agar juga BFS bisa di print bukan cuman DFS
aja

void printDFS(Graph &G, adrNode N); // ditambahkan di soal no 2
void printBFS(Graph &G, adrNode N); // ditambahkan di soal no 3

#endif

```

Code *Graph.cpp*

```

#include "graph.h"
#include <queue>
#include <stack>

void createGraph(Graph &G)
{
    G.first = NULL;
}

adrNode AllocatedNode(infoGraph X)
{
    adrNode P = new ElmNode;
    P->info = X;
    P->visited = 0;
}

```

```

P->firstEdge = NULL;
P->next = NULL;
return P;
}

adrEdge AllocatedEdge(adrNode N)
{
    adrEdge P = new ElmEdge;
    P->node = N;
    P->next = NULL;
    return P;
}

void InsertNode(Graph &G, infoGraph X)
{
    adrNode P = AllocatedNode(X);
    P->next = G.first;
    G.first = P;
}

adrNode findNode(Graph G, infoGraph X)
{
    adrNode P = G.first;
    while (P != NULL)
    {
        if (P->info == X)
            return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(Graph &G, infoGraph A, infoGraph B)
{
    adrNode N1 = findNode(G, A);
    adrNode N2 = findNode(G, B);

    if (N1 == NULL || N2 == NULL)
    {
        cout << "Node tidak ditemukan!" << endl;
        return;
    }

    // buat edge dari N1 ke N2
    adrEdge E1 = AllocatedEdge(N2);
    E1->next = N1->firstEdge;
    N1->firstEdge = E1;
}

```

```

// karena undirected -> buat edge balik
adrEdge E2 = AllocatedEdge(N1);
E2->next = N2->firstEdge;
N2->firstEdge = E2;
}

void PrintInfoGraph(Graph G)
{
    adrNode P = G.first;
    while (P != NULL)
    {
        cout << P->info << " -> ";
        adrEdge E = P->firstEdge;
        while (E != NULL)
        {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

void ResetVisited(Graph &G)
{
    adrNode P = G.first;
    while (P != NULL)
    {
        P->visited = 0;
        P = P->next;
    }
}

// ditambahkan di soal no 2
void printDFS(Graph &G, adrNode N)
{
    if (N == NULL)
        return;

    N->visited = 1;
    cout << N->info << " ";

    adrEdge E = N->firstEdge;
    while (E != NULL)
    {
        if (E->node->visited == 0)
        {
            printDFS(G, E->node);
        }
    }
}

```

```

    }
    E = E->next;
}
}

// ditambahkan di soal no 3
void printBFS(Graph &G, adrNode N)
{
    if (N == NULL)
        return;

    queue<adrNode> Q;
    Q.push(N);

    while (!Q.empty())
    {
        adrNode curr = Q.front();
        Q.pop();

        if (curr->visited == 0)
        {
            curr->visited = 1;
            cout << curr->info << " ";

            adrEdge E = curr->firstEdge;
            while (E != NULL)
            {
                if (E->node->visited == 0)
                {
                    Q.push(E->node);
                }
                E = E->next;
            }
        }
    }
}
}

```

Code *mainGraph.cpp*

```

#include "graph.h"
#include "graph.cpp"
#include <iostream>
using namespace std;

int main() {
    Graph G;
    createGraph(G);
}

```

```

InsertNode(G, 'A');
InsertNode(G, 'B');
InsertNode(G, 'C');
InsertNode(G, 'D');
InsertNode(G, 'E');
InsertNode(G, 'F');
InsertNode(G, 'G');
InsertNode(G, 'H');

ConnectNode(G, 'A', 'B');
ConnectNode(G, 'A', 'C');

ConnectNode(G, 'B', 'D');
ConnectNode(G, 'B', 'E');

ConnectNode(G, 'C', 'F');
ConnectNode(G, 'C', 'G');

ConnectNode(G, 'D', 'H');
ConnectNode(G, 'E', 'H');
ConnectNode(G, 'F', 'H');
ConnectNode(G, 'G', 'H');

cout << "=== Struktur Graph ===" << endl;
PrintInfoGraph(G);

cout << "\n=== Traversal DFS dari node A ===" << endl;
ResetVisited(G);
printDFS(G, findNode(G, 'A'));

cout << "\n\n=== Traversal BFS dari node A ===" << endl;
ResetVisited(G);
printBFS(G, findNode(G, 'A'));

cout << endl;
return 0;
}

```

Screenshots Output



```

PS D:\Cpp\laprak 12> cd "d:\Cpp\laprak 12\" ; if ($?) { g++ mainGraph.cpp -o mainGraph } ; if ($?) { .\mainGraph }
=== Struktur Graph ===
H -> G F E D
G -> H C
F -> H C
E -> H B
D -> H B
C -> G F A
B -> E D A
A -> C B

=== Traversal DFS dari node A ===
A C G H F E B D

=== Traversal BFS dari node A ===
A C B G F E D H
PS D:\Cpp\laprak 12>

```

Deskripsi:

Program ini dibuat untuk mempelajari struktur data Graph tidak berarah menggunakan metode pointer. Di sini, bisa membuat titik simpul dan menghubungkannya satu sama lain. Program juga dilengkapi dengan fitur pencarian DFS dan BFS untuk menelusuri data secara berurutan.

D. Kesimpulan

Program ini dirancang untuk membangun struktur Graph tidak berarah dengan menghubungkan node melalui garis penghubung atau edge menggunakan sistem pointer. Setiap titik dapat menyimpan informasi dan mencatat apakah ia sudah dikunjungi atau belum, sementara garis penghubungnya berfungsi sebagai jembatan antar data. Selain bisa menambah dan menghubungkan titik-titik tersebut, program ini dilengkapi dengan metode DFS untuk menelusuri jalur hingga ke bagian terdalam dan BFS untuk memeriksa titik-titik terdekat secara merata. Dengan adanya program ini, hubungan data yang kompleks dapat dikelola dengan lebih rapi, sehingga proses pemetaan dan pencarian informasi menjadi jauh lebih mudah dipahami.

E. Referensi

https://en.wikipedia.org/wiki/Graph_theory#:~:text=In%20mathematics%20and%20computer%20science,of%20study%20in%20discrete%20mathematics.

<https://fikti.umsu.ac.id/graph-pengertian-fungsi-dan-jenisnya/>